

A STEP-BY-STEP TUTORIAL WITH CODE AND OUTPUT

feynsage

*Feynman integrals in SageMath,
from graphs to numbers*

Kazi Abu Rousan

School of Physical Sciences, NISER, Bhubaneswar

Source code and install: github.com/aburousan/feynsage

HOW TO READ THIS TUTORIAL

Every grey box marked **In [n]** is real code. It was run, in this order, in one SageMath session (as in a Jupyter notebook) and the box under it marked **Out [n]** is what came out: printed text, the value of the last line (typeset from Sage's own LaTeX) or a figure. Nothing in the output boxes was typed by hand. To run the same code yourself, see Chapter 1.

The notation is the one of the lecture note *Feynman integrals at one loop* (sir's lecture at NISER, 2026): $D = 4 - 2\epsilon$, Feynman parameters x_i , the graph polynomials $\mathcal{U}$ and $\mathcal{F}$.

BEYOND THE LECTURE

The lecture covered the tadpole and the bubble, IBP, differential equations, Feynman parameters, the graph polynomials $\mathcal{U}$ and $\mathcal{F}$, Mellin-Barnes, sector decomposition, expansion by regions and Kira. feynsage also does things the lecture did not cover. Each of them is explained here from the start, in an amber box like this one, before it is used.

I wrote feynsage in 2024 for my MSc project. After sir's talk it was refined, checked against the lecture note and against Package-X, LoopTools, FeynCalc, LiteRed and Kira, and published at github.com/aburousan/feynsage under the MIT licence.

Contents

1	Getting feynsage	3
1.1	What you need	3
1.2	Download	3
1.3	Install	3
1.4	Check and start	3
1.5	The other material	3
2	First steps: the tadpole and the bubble	4
2.1	Ask a function what it does	4
2.2	The tadpole A_0	4
2.3	The bubble B_0 and its threshold	5
2.4	An exact identity	5
3	One loop with numerators, the Package-X way	6
3.1	A vector numerator, checked by hand	6
3.2	Triangles and boxes	7
3.3	Soft and collinear poles	8
3.4	When the Gram determinant vanishes	8
4	Diagrams as graphs: $\mathcal{U}$ and $\mathcal{F}$	9
4.1	A diagram in one line	9
4.2	The kite: spanning trees	9
4.3	The kite: 2-forests	10
4.4	Three independent checks	11
4.5	The two-loop vertex of the note	12
5	Sectors	13
5.1	Index vectors	13
5.2	What a sector is	13
5.3	Why a missing line is drawn shrunk to a point	13
5.4	All sectors with four and three lines	14
5.5	Zero sectors	15
5.6	Why sectors matter	16
5.7	The same sector under another name	17
6	Integration by parts and master integrals	19
6.1	The bubble family	19
6.2	Reduction to master integrals	19
6.3	A two-loop number from two masters	20
6.4	Three reducers, one answer	20
6.5	Seeds	21
6.6	Counting masters without reducing	21
7	Dirac traces with FORM	22
7.1	A whole cross section: $e^+e^- \rightarrow \mu^+\mu^-$	22
8	Where to go next	23

1 Getting feynsage

1.1 What you need

- **SageMath** 10.7 or newer (tested with 10.7 and 10.9). It is free. The install script below finds it or installs it.
- **FORM**, only for the Dirac traces of Chapter 7. The script installs it too.
- **A C compiler** (optional). With one, the finite-field reducer compiles a small kernel the first time it runs. Without one it runs in pure Python with the same answers, about ten times slower.
- macOS (Apple silicon or Intel), Linux, or Windows through WSL. SageMath does not run on Windows itself.

1.2 Download

The code is at github.com/aburousan/feynsage. Either clone it with git

```
git clone https://github.com/aburousan/feynsage.git
cd feynsage
```

or, without git, open the page in a browser, press **Code** and then **Download ZIP**, unpack it and open a terminal in the unpacked folder `feynsage-main`.

1.3 Install

From inside the folder:

```
./install.sh          # finds or installs SageMath and FORM, then installs feynsage
./install.sh --test    # the same, then runs every test (a few minutes)
./install.sh --no-deps # installs feynsage only, does not touch SageMath or FORM
```

What the script does, step by step:

1. It looks for SageMath: the `sage` command, a conda environment called `sage`, or the macOS app. If there is none it installs it (with Homebrew on macOS, or as the conda environment `sage` from conda-forge on Linux and WSL). This is a large download.
2. It looks for FORM and installs it if it is missing (Homebrew on macOS, the official release binary into `~/local/bin` on Linux).
3. It installs feynsage into Sage's own Python with `sage -pip`. On the macOS app this is a user install, so the signed app is never changed.
4. It imports the package once to check it and to compile the C kernel.

If you prefer to do it by hand, the third step alone is

```
sage -pip install --user --no-deps .
```

1.4 Check and start

```
sage -c "import feynsage; print(feynsage.__file__)" # where it was installed
sage -n jupyterlab                               # notebooks, choose the SageMath kernel
```

Then open `tutorials/01_first_steps.ipynb`. To update later: `git pull` and `./install.sh --no-deps`.

1.5 The other material

- `tutorials/`: five short Jupyter notebooks with exercises and solutions, the same steps as this tutorial.
- `examples/feynsage_walkthrough.ipynb`: a longer tour, with the neutral pion lifetime and the QED of Chluba's thesis.
- `docs/DETAILS.md`: every module, every check against other programs, speed and limits.
- `docs/tutorial/`: this tutorial. `sage docs/tutorial/run_cells.sage` runs all its cells again and `typst compile docs/tutorial/feynsage_tutorial.typ` makes the PDF.

2 First steps: the tadpole and the bubble

Load the package. Everything below is in one namespace, like the cells of a notebook.

```
In [1]: from feynsage import *
```

2.1 Ask a function what it does

Every public function carries its own explanation. `info(f)` prints it and most functions also take `explain=True`.

```
In [2]: info(A0)
```

```
A0
--
Tadpole A0(m) = m^2 (1/eps + log(mu^2/m^2) + 1) in the Package-X normalisation; A0(0) = 0.
Output: a symbolic expression in eps (the 1/eps UV pole) and mu (the renormalisation
scale).
```

2.2 The tadpole A_0

The simplest loop integral has one propagator. `var` makes a symbol in Sage.

```
In [3]: m = var('m')
a = A0(m)
a
```

```
Out[3]:
```

$$m^2 \left(\frac{1}{\epsilon} + \log \left(\frac{\mu^2}{m^2} \right) + 1 \right)$$

The $1/\epsilon$ is the ultraviolet pole and μ is the renormalisation scale. Three small functions pull the pieces apart: `uv_part` (the coefficient of $1/\epsilon$), `pole_parts` (the coefficients of $1/\epsilon^2$ and $1/\epsilon$) and `finite_part` (the ϵ^0 part, with the value of μ as an optional second argument).

```
In [4]: print("pole:          ", uv_part(a))
print("finite part:       ", finite_part(a))
print("finite, mu = m:    ", finite_part(a, m))
```

```
pole:          m^2
finite part:    m^2*log(mu^2/m^2) + m^2
finite, mu = m: m^2
```

BEYOND THE LECTURE: THE NORMALISATION OF PACKAGE-X

The note works in Euclidean space with $[d\ell] = d^D \ell / \pi^{D/2}$ and propagators $1/(\ell^2 + m^2)$. The one-loop functions of feynsage (A_0 , B_0 , C_0 , D_0 and `loop`) follow instead Package-X and LoopTools, the two programs most people compare with:

$$A_0(m) = \mu^{2\epsilon} e^{\epsilon \gamma_E} \int \frac{d^D \ell_M}{i \pi^{D/2}} \frac{1}{\ell_M^2 - m^2 + i0}$$

in Minkowski space. The two are the same integral seen two ways. The Wick rotation gives $d^D \ell_M = i d^D \ell_E$, which cancels the $1/i$, and $\ell_M^2 - m^2 = -(\ell_E^2 + m^2)$, which gives one minus sign for each propagator. The factor $e^{\epsilon \gamma_E}$ removes the γ_E that every loop produces. So A_0 is minus the note's tadpole times $e^{\epsilon \gamma_E}$:

```
In [9]: from feynsage.oneloop import tadpole, expand_eps, D
note = expand_eps(tadpole(1, 1), 0)      # the note: e^(eps gamma_E) Int [dl] 1/(l^2 +
1), Euclidean
px = A0(1).subs(mu=1)                   # Package-X normalisation, Minkowski, m = mu =
1
print("note's tadpole:", note)
print("A0(1):          ", px)
print("sum:             ", (note + px).simplify_full())
```

```
note's tadpole: -(eps + 1)/eps
A0(1):          1/eps + 1
sum:            0
```

2.3 The bubble B_0 and its threshold

Two propagators, (ℓ, m_1) and $(\ell + p, m_2)$ with $p^2 = s$. Λ is the function that Package-X calls `DiscB`, the logarithm that carries the threshold.

```
In [5]: s = var('s')
        B0(s, m, m)
```

```
Out[5]:
```

$$\frac{1}{\epsilon} + \Lambda(s; m, m) + \log\left(\frac{\mu^2}{m^2}\right) + 2$$

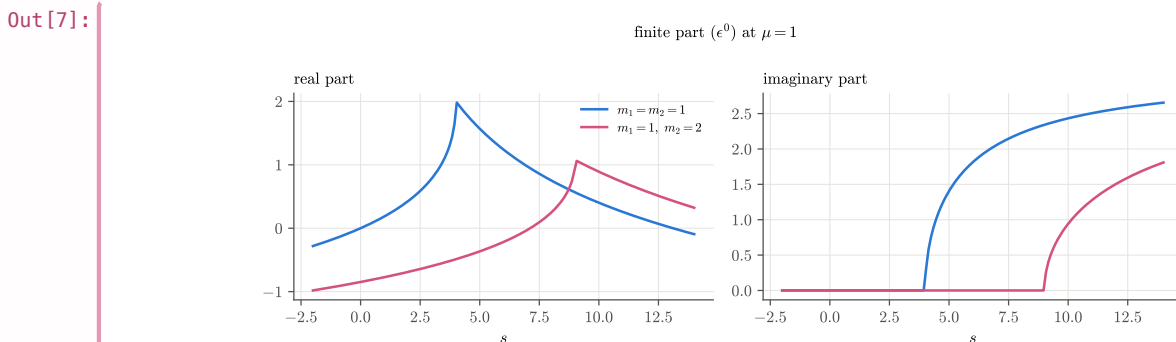
Below the threshold $s = 4m^2$ the bubble is real. Above it the two particles in the loop can be real, so it gets an imaginary part. The note (Minkowski chapter) finds $\text{Im } B_0 = \pi\beta$ with $\beta = \sqrt{1 - 4m^2/s}$. Check it with $m = \mu = 1$:

```
In [6]: for sv in [1, 3, 5, 10]:
        val = finite_part(B0(sv, 1, 1), 1).n(digits=12)
        beta = sqrt(1 - 4/sv) if sv > 4 else 0
        print("s = %2d    B0 finite = %-38s pi*beta = %.12f" % (sv, val, (pi*beta).n()))
```

s = 1	B0 finite = 0.186200635766 + 2.09199576156e-26*I	pi*beta = 0.000000000000
s = 3	B0 finite = 0.790800423844 + 1.41839915231e-25*I	pi*beta = 0.000000000000
s = 5	B0 finite = 1.56959105904 + 1.40496294621*I	pi*beta = 1.404962946208
s = 10	B0 finite = 0.401668519244 + 2.43346720558*I	pi*beta = 2.433467205584

Below threshold the imaginary part is zero (the 10^{-26} is rounding), above it is $\pi\beta$ to every digit. A plot shows the kink at each threshold, $s = (m_1 + m_2)^2 = 4$ and 9. `quick_plot` draws the real part as a full line and the imaginary part dashed.

```
In [7]: from feynsage.plotting import quick_plot, set_theme
        set_theme()
        fig = quick_plot([B0(s, 1, 1), B0(s, 1, 2)], (s, -2, 14),
                        labels=["$m_1 = m_2 = 1$", "$m_1 = 1, m_2 = 2$"])
```



2.4 An exact identity

At $s = 0$ with equal masses the bubble is the derivative of the tadpole with respect to m^2 , which gives $A_0(m) = m^2(B_0(0; m, m) + 1)$. Sage checks it exactly:

```
In [8]: (A0(m) - m^2*(B0(0, m, m) + 1)).simplify_full()
```

```
Out[8]: 0
```

3 One loop with numerators, the Package-X way

BEYOND THE LECTURE: TENSOR INTEGRALS AND THEIR REDUCTION

In a real amplitude the loop momentum also appears upstairs, from the Dirac algebra, as ℓ^μ , $\ell^\mu \ell^\nu$, $\ell \cdot p$ and so on. Passarino and Veltman (1979) showed that every such **tensor integral** is a combination of the scalar integrals A_0, B_0, C_0, D_0 . The recipe:

1. Write the most general answer allowed by Lorentz symmetry. For the bubble only p^μ and $g^{\mu\nu}$ exist, so $\int \ell^\mu / (D_1 D_2) = p^\mu B_1$ and $\int \ell^\mu \ell^\nu / (D_1 D_2) = g^{\mu\nu} B_{00} + p^\mu p^\nu B_{11}$.
2. Contract both sides with every vector and with $g_{\mu\nu}$. On the left, write each $\ell \cdot p$ and ℓ^2 through the propagators: then a propagator cancels and a simpler integral is left.
3. Solve the linear system for the coefficients $B_1, B_{00}, B_{11}, \dots$

The matrix of this system is built from the scalar products of the external momenta, the **Gram matrix** $G_{ij} = p_i \cdot p_j$. Solving it divides by $\det G$. `loop(numerator, propagators, kin)` does all of this exactly, with $D = 4 - 2\epsilon$ kept until the end (a D in a coefficient times a $1/\epsilon$ pole leaves a finite **rational term**).

`loop(numerator, propagator, ...)` takes strings. Each propagator is [momentum, mass] and kin gives the scalar products. The answer is one line per tensor structure:

```
In [10]: loop("1", ["l", "m1"], ["l + p", "m2"], kin={"p^2": "s"})
```

```
Out[10]: 1:  1/epsilon - (m1^2 - m2^2 + s)/(2s) * log(m1^2/m2^2)
          + Lambda(s; m1, m2) + log(mu^2/m2^2)
          + 2
```

3.1 A vector numerator, checked by hand

With ℓ^μ upstairs the only vector the answer can point along is p^μ :

```
In [11]: loop("l^mu", ["l", "m1"], ["l + p", "m2"], kin={"p^2": "s"})
```

```
Out[11]: p^mu: -1/(2*epsilon) + (m1^2 - m2^2 + s)/(4s^2) * log(m1^2/m2^2)
            - (m1^2 - m2^2 + s)/(2s) * Lambda(s; m1, m2) + m1^2/(2s) * log(mu^2/m1^2)
            - (m1^2 + s)/(2s) * log(mu^2/m2^2) - (m1^2 - m2^2 + 2s)/(2s)
```

THE SAME COEFFICIENT BY HAND

Contract with p_μ and use $2\ell \cdot p = D_2 - D_1 - (s + m_1^2 - m_2^2)$, where $D_1 = \ell^2 - m_1^2$ and $D_2 = (\ell + p)^2 - m_2^2$. Each D_i upstairs cancels one propagator and leaves a tadpole:

$$B_1 = \frac{1}{2s} [A_0(m_1) - A_0(m_2) - (s + m_1^2 - m_2^2) B_0(s; m_1, m_2)]$$

`PVB(0, 1, ...)` is Package-X's `PVB[0, 1, ...]`, that is B_1 . Compare the two at a point above threshold:

```
In [12]: sv, a1, a2 = 10, 1, 2
hand = (A0(a1) - A0(a2) - (sv + a1^2 - a2^2)*B0(sv, a1, a2))/(2*sv)
pv = PVB(0, 1, sv, a1, a2) # Package-X's PVB[0, 1, ...] = B1
print("poles: ", uv_part(hand), uv_part(pv))
print("finite parts: ", finite_part(hand, 1).n(), " ", finite_part(pv, 1).n())
```

```
poles:          -1/2 -1/2
finite parts:  -0.184578706662452 - 0.329867228626928*I      -0.184578706662452 -
0.329867228626928*I
```

Rank two works the same way. The structures are $g^{\mu\nu}$ and $p^\mu p^\nu$:

```
In [13]: loop("l^mu l^nu", ["l", "m"], ["l + p", "m"], kin={"p^2": "s"})
```

```
Out[13]:
```

$$g^{\mu\nu} : \frac{1}{\epsilon} \left(\frac{1}{2} m^2 - \frac{1}{12} s \right) + \left(\frac{1}{3} m^2 - \frac{1}{12} s \right) \Lambda(s; m, m)$$

$$+ \left(\frac{1}{2} m^2 - \frac{1}{12} s \right) \log \left(\frac{\mu^2}{m^2} \right) + \frac{7}{6} m^2 - \frac{2}{9} s$$

$$[6pt] p^\mu p^\nu : \frac{1}{3\epsilon} - \frac{m^2 - s}{3s} \Lambda(s; m, m)$$

$$+ \frac{1}{3} \log \left(\frac{\mu^2}{m^2} \right) - \frac{12m^2 - 13s}{18s}$$

3.2 Triangles and boxes

BEYOND THE LECTURE: $\$C_0\$$ AND $\$D_0\$$ AS NUMBERS AND AS FORMULAS

The scalar triangle and box are the next functions after B_0 . With Feynman parameters (as in the note) the triangle is a two-dimensional integral over the simplex $x_0 + x_1 + x_2 = 1$,

$$C_0 = - \int d^3x \delta(1 - x_0 - x_1 - x_2) \frac{1}{\mathcal{F} - i0}, \quad \mathcal{F} = \sum_i x_i m_i^2 - \sum_{i < j} x_i x_j s_{ij}$$

where s_{ij} is the square of the momentum flowing between lines i and j (in Minkowski space, so $\mathcal{F}$ carries the minus sign). The minus sign in front is the Wick rotation again. 't Hooft, Veltman and later Denner integrated it in closed form: a sum of logarithms and dilogarithms Li_2 whose arguments are roots of quadratic equations. The hard part is the $+i0$: it decides on which side of each branch cut every logarithm sits. feynsage keeps every quantity both as an exact expression and as an 80-digit number with a tiny $+i0$, lets the number pick the branch and compares the two at the end.

A finite triangle stays a symbol until a number is asked for. `c0_numeric` integrates the Feynman-parameter form numerically, a second and independent way. `explicit` writes it in logarithms and dilogarithms:

```
In [15]: c = C0(1, 2, 3, 1, 2, 3)
print(c, "=", c.n(digits=30))
print("direct numerical integration:", c0_numeric(1, 2, 3, 1, 2, 3))
print("as logs and dilogs:", len(str(explicit(c))), "characters, value",
explicit(c).n(digits=20).real())
```

```
C0(1, 2, 3, 1, 2, 3) = -0.142398839895237347030644540980
direct numerical integration: (-0.1423988398952359+0j)
as logs and dilogs: 1206 characters, value -0.14239883989523734703
```

To see that nothing is hidden, do the integral of the box above yourself with SciPy at a point where every invariant is spacelike (there $\mathcal{F} > 0$ and no $i0$ is needed):

```
In [16]: from scipy.integrate import dblquad
s1v, s12v, s2v, m0, m1, m2 = -1.0, -2.0, -3.0, 1.0, 2.0, 3.0      # all invariants
spacelike: F > 0
def Fpar(x1, x2):
    x0 = 1 - x1 - x2      # x0, x1, x2 on the lines (l, m0), (l + p1,
m1), (l + p2, m2)
    return x0*m0**2 + x1*m1**2 + x2*m2**2 - x0*x1*s1v - x0*x2*s2v - x1*x2*s12v
by_hand = -dblquad(lambda x2, x1: 1/Fpar(x1, x2), 0, 1, 0, lambda x1: 1 - x1, epsabs=1e-13)
[0]
print("by hand: ", by_hand)
print("feynsage: ", C0(-1, -2, -3, 1, 2, 3).n(digits=16))
```

```
by hand: -0.11122756310943065
feynsage: -0.1112275631094306
```

A box, in the LoopTools order of arguments $D_0(s_1, s_2, s_3, s_4; s_{12}, s_{23}; m_0, \dots, m_3)$:

```
In [17]: d = D0(-1, -2, -3, -4, -5, -6, 1, 2, 3, 4)
print(d.n(digits=25))
print(d0_numeric(-1, -2, -3, -4, -5, -6, 1, 2, 3, 4))

0.003369962787211075225506485
(0.0033699627872110765+0j)
```

3.3 Soft and collinear poles

BEYOND THE LECTURE: INFRARED DIVERGENCES

The ultraviolet pole comes from large loop momenta. With massless particles there is a second kind, the **infrared** divergences: a massless line can carry a very small momentum (**soft**) or move exactly along a light-like external leg (**collinear**). Both make propagators vanish inside the integration region. In dimensional regularisation they also show up as poles in ϵ , and when soft and collinear meet they give $1/\epsilon^2$. In a physical cross section they cancel against the emission of real soft photons or gluons (the Bloch-Nordsieck and KLN theorems), which is why they must be kept exactly. feynsage has the six divergent triangles and sixteen divergent boxes of Ellis and Zanderighi (2008) built in.

A massless triangle with two light-like legs:

```
In [18]: pole_parts(C0(0, s, 0, 0, 0, 0))
```

```
Out[18]:
```

$$\left(\frac{1}{s}, -\frac{\ln(-s-i0)}{s} + \frac{\log(\mu^2)}{s} \right)$$

Where does the $1/\epsilon^2$ come from? Build the same triangle as a graph. Of its three 2-forests only one carries momentum, because the other two cut off a single light-like leg with $p_i^2 = 0$:

```
In [19]: tri = graph("A-B, B-C, C-A", {"A": "p1 + p2", "B": "-p1", "C": "-p2"},
kin={"p1^2": 0, "p2^2": 0, "p1.p2": "Q2/2"}, euclidean=True)
print("U =", tri.U(), "      F =", tri.F())
ep = var('eps')
dirichlet = gamma(1 + ep)*gamma(-ep)^2/gamma(1 - 2*ep)
(exp(ep*euler_gamma)*dirichlet).series(ep, 1).truncate()

U = x1 + x2 + x3      F = Q2*x1*x3
```

```
Out[19]:
```

$$-\frac{1}{12}\pi^2 + \frac{1}{\epsilon^2}$$

So $\mathcal{F} = Q^2 x_1 x_3$ and the parametric integrand contains $(x_1 x_3)^{-1-\epsilon}$. Each of $\int_0 dx_1 x_1^{-1-\epsilon}$ and $\int_0 dx_3 x_3^{-1-\epsilon}$ is $-1/\epsilon$ near its end point: one is the collinear region of one leg, the other of the other leg, and the corner where both vanish is the soft region. Together they give the Dirichlet integral $\Gamma(1+\epsilon)\Gamma(-\epsilon)^2/\Gamma(1-2\epsilon)$, whose expansion (with $e^{\epsilon\gamma_E}$) is the last output: $1/\epsilon^2 - \pi^2/12$, as in the note.

3.4 When the Gram determinant vanishes

BEYOND THE LECTURE: A ZERO GRAM DETERMINANT

The tensor reduction divides by $\det G$. For a triangle with $p_1^2 = s_1$, $p_2^2 = s_2$ and $(p_1 - p_2)^2 = s_{12}$:

```
In [14]: s1, s2, s12 = var('s1 s2 s12')
p1p2 = (s1 + s2 - s12)/2 # p1.p2 from s12 = (p1 - p2)^2
Gram = matrix([s1, p1p2], [p1p2, s2])
print("det Gram =", Gram.det().expand())
```

```
print("at the g-2 point s1 = s2 = m^2, q^2 = s12 = 0:", Gram.det().subs(s1=m^2, s2=m^2, s12=0))
```

```
det Gram = -1/4*s1^2 + 1/2*s1*s12 - 1/4*s12^2 + 1/2*s1*s2 + 1/2*s12*s2 - 1/4*s2^2
at the g-2 point s1 = s2 = m^2, q^2 = s12 = 0: 0
```

At the $g - 2$ point of QED ($p_1 = p_2$, photon momentum $q^2 = 0$) the determinant is exactly zero, so the textbook reduction fails there. feynsage then does not divide at all: the lines with the same momentum are grouped and the coefficients come straight from the Feynman-parameter integral.

```
In [20]: print("C1 =", PVC(0, 1, 0, m^2, 0, m^2, 0, m, m))
print("C00 =", PVC(1, 0, 0, m^2, 0, m^2, 0, m, m).log_expand().expand())

C1 = 1/2/m^2
C00 = 1/4/eps - 1/2*log(m) + 1/2*log(mu) + 1/4
```

Package-X gives $C_1 = 1/(2m^2)$ and $C_{00} = 1/4 + (1/\varepsilon + \log \mu^2/m^2)/4$, the same.

4 Diagrams as graphs: $\mathcal{U}$ and $\mathcal{F}$

This chapter follows the second part of the lecture (graph polynomials).

4.1 A diagram in one line

graph(edges, legs, kin):

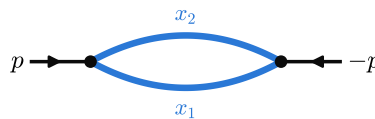
- edges: the lines, "A-B" between two vertices or "A-B:m" for a line with mass m . **The order of the lines is the numbering** $x_1, x_2, \dots$ of $\mathcal{U}$ and $\mathcal{F}$.
- legs: the momentum **entering** at each vertex. They must add up to zero.
- kin: the scalar products of the external momenta. euclidean=True for Euclidean signs.

The number of loops is $L = N - V + 1$ (lines minus vertices plus one).

```
In [21]: bub = graph("A-B:ma, A-B:mb", {"A": "p", "B": "-p"}, kin={"p^2": "pp"}, euclidean=True)
print("lines N =", bub.N, " vertices V =", bub.V, " loops L = N - V + 1 =", bub.L)
fig = bub.plot(figsize=(2.6, 2.6))
```

```
lines N = 2   vertices V = 2   loops L = N - V + 1 = 1
```

Out[21]:



In the pictures a massive line is thick and blue and a massless line thin and black. Each line carries its x_i .

4.2 The kite: spanning trees

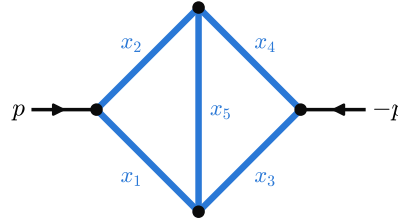
The two-loop kite of the note is ready-made. It has four vertices: L and R where p enters and leaves, T and B at the top and the bottom.

```
In [22]: from feynsage.plotting import draw_panels, draw_sectors, draw_graph
kite = diagram("kite")
```

```
print(kite.lines)
fig = kite.plot(figsize=(2.8, 2.8))
```

```
[('L', 'B', '0'), ('L', 'T', '0'), ('B', 'R', '0'), ('T', 'R', '0'), ('T', 'B', '0')]
```

Out[22]:

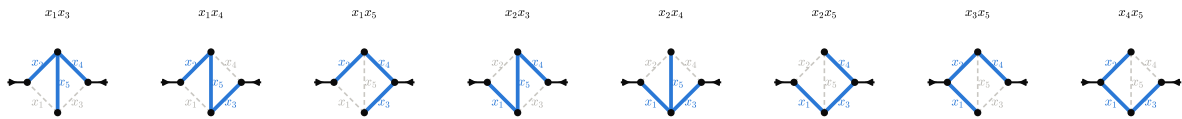


A **spanning tree** touches every vertex and has no closed loop. For L loops we must remove L lines. `spanning_trees()` lists the removed lines (counted from 0). In the pictures the removed lines are grey and dashed:

```
In [23]: xs = lambda rem: "$" + "".join("x_{" + str(i + 1) + "}" for i in rem) + "$"
trees = kite.spanning_trees()
print(len(trees), "spanning trees; removed lines (from 0):", trees)
fig = draw_panels(kite, trees, titles=[xs(r) for r in trees], momenta=False, ncols=8,
size=1.5)
```

```
8 spanning trees; removed lines (from 0): [(0, 2), (0, 3), (0, 4), (1, 2), (1, 3), (1, 4),
(2, 4), (3, 4)]
```

Out[23]:



$\mathcal{U}$ is the sum over spanning trees of the product of the x_i of the **removed** lines:

```
In [24]: kite.U()
```

```
Out[24]: x1x3 + x2x3 + x1x4 + x2x4 + x1x5 + x2x5 + x3x5 + x4x5
```

4.3 The kite: 2-forests

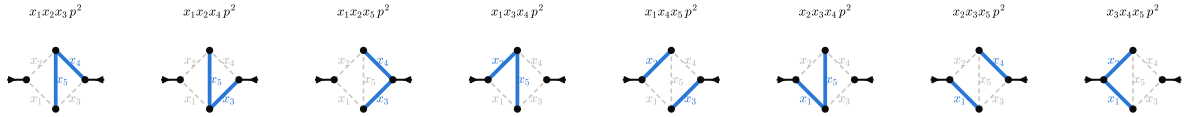
Remove one more line, $L + 1$ in all, and the graph falls into two trees. If the momentum p has to flow from one tree to the other, the 2-forest gives (product of the removed x_i) $\times p^2$. If L and R sit in the same tree, no momentum flows between the trees and the term is zero.

```
In [25]: def entering(comp):
    tot = {}
    for v in comp:
        for k, c in kite.ext.get(v, {}).items():
            tot[k] = tot.get(k, 0) + c
    return {k: c for k, c in tot.items() if c}

forests = kite.two_forests()
carry = [(r, c) for r, c in forests if entering(c)]
print(len(forests), "2-forests,", len(carry), "of them carry the momentum p")
fig = draw_panels(kite, [r for r, c in carry], titles=[xs(r) + "$\\,p^2$" for r, c in
carry],
momenta=False, ncols=8, size=1.5)
```

```
10 2-forests, 8 of them carry the momentum p
```

Out[25]:



In Minkowski space $\mathcal{F} = -\mathcal{F}_0 + \mathcal{U} \sum x_i m_i^2$, hence the $-s$:

In [26]: `kite.F()`Out[26]: $(-s)x_1x_2x_3 + (-s)x_1x_2x_4 + (-s)x_1x_3x_4 + (-s)x_2x_3x_4 + (-s)x_1x_2x_5 + (-s)x_2x_3x_5 + (-s)x_1x_4x_5 + (-s)x_3x_4x_5$

4.4 Three independent checks

feynsage finds the same polynomials in two more ways and compares. The matrix method of the lecture ($\mathcal{U} = \det M$ after completing the square) needs momenta on the lines. The third way, Kirchhoff's theorem, was not in the lecture.

BEYOND THE LECTURE: KIRCHHOFF'S MATRIX-TREE THEOREM

Kirchhoff (1847, for electric circuits) showed that spanning trees can be counted with a determinant. Give line i the weight $1/x_i$ and build the **Laplacian** of the graph: on the diagonal, the sum of the weights of the lines at that vertex, and off the diagonal, minus the weight of the line between the two vertices. Strike out one row and the same column. The determinant of what is left is the sum over all spanning trees T of $\prod_{i \in T} 1/x_i$. Multiplying by $\prod_i x_i$ turns "lines in the tree" into "lines removed", which is exactly $\mathcal{U}$. For the kite (rows and columns in the order of the vertices printed):

```
In [28]: X = [SR.var('x%d' % (i + 1)) for i in range(kite.N)]
idx = {v: n for n, v in enumerate(kite.vertices)}
Lap = matrix(SR, kite.V, kite.V)
for i, (u, v, _) in enumerate(kite.lines): # line i between u and v, weight 1/x_i
    a, b = idx[u], idx[v]
    Lap[a, a] += 1/X[i]; Lap[b, b] += 1/X[i]; Lap[a, b] -= 1/X[i]; Lap[b, a] -= 1/X[i]
print("vertices in this order:", kite.vertices)
Lap
vertices in this order: ['B', 'L', 'R', 'T']
```

Out[28]:

$$\begin{pmatrix} \frac{1}{x_1} + \frac{1}{x_3} + \frac{1}{x_5} & -\frac{1}{x_1} & -\frac{1}{x_3} & -\frac{1}{x_5} \\ -\frac{1}{x_1} & \frac{1}{x_1} + \frac{1}{x_2} & 0 & -\frac{1}{x_2} \\ -\frac{1}{x_3} & 0 & \frac{1}{x_3} + \frac{1}{x_4} & -\frac{1}{x_4} \\ -\frac{1}{x_5} & -\frac{1}{x_2} & -\frac{1}{x_4} & \frac{1}{x_2} + \frac{1}{x_4} + \frac{1}{x_5} \end{pmatrix}$$

```
In [29]: reduced = Lap[1:, 1:] # strike out one row and column
U_from_det = (prod(X) * reduced.det()).expand()
print("prod(x) * det(reduced Laplacian) =", U_from_det)
print("equal to U from the trees:", bool(U_from_det == SR(str(kite.U())).expand()))
```

```
prod(x) * det(reduced Laplacian) = x1*x3 + x2*x3 + x1*x4 + x2*x4 + x1*x5 + x2*x5 + x3*x5 + x4*x5
equal to U from the trees: True
```

BEYOND THE LECTURE: MOMENTA FROM THE GRAPH

To use the matrix method, or to build an integral family, every line needs a momentum. `family()` does it like this: pick one spanning tree; every line **not** in it (a **chord**) gets its own loop momentum, which flows around the one closed loop that the chord makes with the tree; the external momenta flow through the tree by momentum conservation at each vertex. There are exactly L chords, so there are L loop momenta.

With momenta on the lines, the three ways can be compared on graphs of one, two and three loops:

```
In [27]: import time
def three_ways(g, name):
    t0 = time.time()
    U, F = g.U(), g.F()
    t1 = time.time()
    props, loops = g.family()
    Um, Fm = IntegralFamily(name, loops, g.kin, props).UF()
    same = g.U_kirchhoff() == U and g.R(str(Um)) == U and g.R(str(Fm)) == F
    print("%-10s L = %d, %d lines, %d trees, %d 2-forests (%.3f s); trees = Kirchhoff = det M: %s"
          % (name, g.L, g.N, len(g.spanning_trees()), len(g.two_forests()), t1 - t0, same))

for name in ["bubble_mass", "kite", "vertex2", "banana3", "ladder3", "triplebox"]:
    three_ways(diagram(name), name)
```

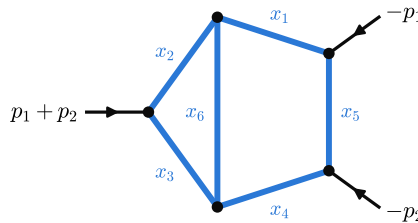
```
bubble_mass L = 1, 2 lines, 2 trees, 1 2-forests (0.000 s); trees = Kirchhoff = det M: True
kite        L = 2, 5 lines, 8 trees, 10 2-forests (0.001 s); trees = Kirchhoff = det M: True
vertex2     L = 2, 6 lines, 11 trees, 19 2-forests (0.001 s); trees = Kirchhoff = det M: True
banana3     L = 3, 4 lines, 4 trees, 1 2-forests (0.000 s); trees = Kirchhoff = det M: True
ladder3     L = 3, 8 lines, 30 trees, 59 2-forests (0.003 s); trees = Kirchhoff = det M: True
triplebox   L = 3, 10 lines, 56 trees, 163 2-forests (0.010 s); trees = Kirchhoff = det M: True
```

4.5 The two-loop vertex of the note

vertex2 is the planar two-loop vertex of the sector-decomposition chapter of the note, with the lines numbered as in the note's picture:

```
In [30]: v2 = diagram("vertex2")
fig = v2.plot(figsize=(3.0, 3.0))
```

Out[30]:



```
In [31]: R = v2.R
x1, x2, x3, x4, x5, x6 = R.gens()
q2 = R.base_ring()('q2')
U_note = (x2 + x3)*(x1 + x4 + x5) + (x1 + x2 + x3 + x4 + x5)*x6
F_note = -q2*(x1*x3*x4 + x1*x2*(x3 + x4) + x2*x3*(x4 + x5) + (x1 + x2)*(x3 + x4)*x6)
print("U as in the note:", v2.U() == U_note, " F as in the note:", v2.F() == F_note)
```

```
U as in the note: True    F as in the note: True
```

5 Sectors

BEYOND THE LECTURE: SECTORS

The lecture used sectors in the reduction of the kite without dwelling on them. They are the bridge between the graph and integration by parts, so this chapter goes slowly: what an index vector is, what a sector is, why a missing line is drawn shrunk to a point, which sectors are zero, and why all this matters for the master integrals.

5.1 Index vectors

Take the kite as a **family** of integrals. `family()` gives every line a momentum (as explained above):

```
In [32]: props, loops = kite.family()
kfam = IntegralFamily("kite", loops, kite.kin, props)
for i, (q, m2) in enumerate(props):
    print("line %d (x%d): D%d = (%s)^2" % (i + 1, i + 1, i + 1,
        " + ".join(("" if c == 1 else "-" if c == -1 else str(c)) + k for k, c in
q.items())).replace("+ -", "-"))

line 1 (x1): D1 = (l1)^2
line 2 (x2): D2 = (-l1 + p)^2
line 3 (x3): D3 = (l2)^2
line 4 (x4): D4 = (-l2 + p)^2
line 5 (x5): D5 = (-l1 + l2)^2
```

Every member of the family is fixed by five integers, one per line:

$$F(a_1, \dots, a_5) = \int d^D \ell_1 d^D \ell_2 \frac{1}{D_1^{a_1} D_2^{a_2} D_3^{a_3} D_4^{a_4} D_5^{a_5}}$$

- $a_i = 1$: line i is an ordinary propagator
- $a_i = 2, 3, \dots$: the same propagator raised to a higher power (a **dot** on the line)
- $a_i = 0$: the propagator is **absent**
- $a_i < 0$: D_i sits upstairs as a **numerator**

5.2 What a sector is

The **sector** of $F(a_1, \dots, a_5)$ is the pattern of lines that are really there: a 1 where $a_i > 0$ and a 0 where $a_i \leq 0$. Dots and numerators do not change the sector:

```
In [33]: def sector(a):
    return tuple(1 if x > 0 else 0 for x in a)

for a in [(1, 1, 1, 1, 1), (2, 1, 1, 1, 3), (1, 1, 1, 1, -2), (1, 1, 1, 1, 0), (0, 1, 1, 0, 1), (0, 2, 1, -1, 1)]:
    print("F%s is in sector %s" % (a, sector(a)))

F(1, 1, 1, 1, 1) is in sector (1, 1, 1, 1, 1)
F(2, 1, 1, 1, 3) is in sector (1, 1, 1, 1, 1)
F(1, 1, 1, 1, -2) is in sector (1, 1, 1, 1, 0)
F(1, 1, 1, 1, 0) is in sector (1, 1, 1, 1, 0)
F(0, 1, 1, 0, 1) is in sector (0, 1, 1, 0, 1)
F(0, 2, 1, -1, 1) is in sector (0, 1, 1, 0, 1)
```

So the sector tells us **which diagram** we are looking at. The dots and numerators only change what sits on its lines. A family with t lines has 2^t sectors, here $2^5 = 32$.

5.3 Why a missing line is drawn shrunk to a point

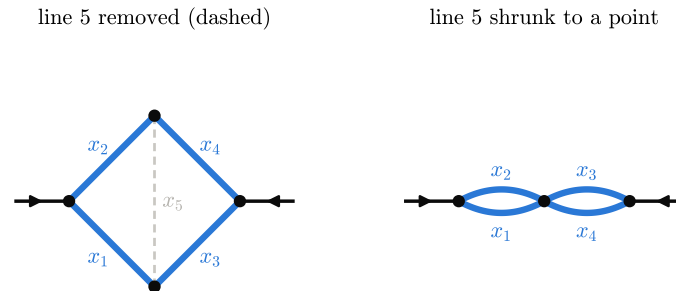
When $a_5 = 0$ the propagator $1/D_5$ is gone, but the integral is still over both loop momenta. Look at the momenta above: without D_5 the remaining four depend on ℓ_1 (lines 1, 2) or on ℓ_2 (lines 3, 4), never on both. So the integral splits into two one-loop bubbles that touch at one point. In the picture this means the two ends of line

5 (the vertices T and B) become **one vertex**: line 5 is shrunk to a point (contracted). It is **not** erased, since an erased line would leave a one-loop graph with four lines.

```
In [34]: import matplotlib.pyplot as plt
h, kept = kite.contract([5])          # the kite without line 5
print("lines kept:", kept, " vertices:", h.vertices)
fig, axes = plt.subplots(1, 2, figsize=(5.2, 2.5))
draw_graph(kite, ax=axes[0], momenta=False, removed=[5], title="line 5 removed (dashed)")
draw_graph(h, ax=axes[1], names=kept, momenta=False, title="line 5 shrunk to a point")
```

```
lines kept: [1, 2, 3, 4]   vertices: ['L', 'R', 'T']
```

Out[34]:



The same holds for the graph polynomials. The parametric formula of a sector is the one of the full family with $x_j = 0$ for every missing line j (an index $a_j = 0$ puts $x_j^{a_j-1}/\Gamma(a_j)$, which becomes $\delta(x_j)$). Sage checks that this is exactly $\mathcal{U}$ and $\mathcal{F}$ of the shrunk graph:

```
In [35]: to_kite = h.R.hom([kite.R.gen(k - 1) for k in kept], kite.R)    # x_k of h -> x_kept[k] of
the kite
x5 = kite.R.gen(4)
print("U(kite) at x5 = 0 = U(kite with line 5 shrunk):", kite.U().subs({x5: 0}) ==
to_kite(h.U()))
print("F(kite) at x5 = 0 = F(kite with line 5 shrunk):", kite.F().subs({x5: 0}) ==
to_kite(h.F()))
print("the same graph with line 5 deleted instead has", len(kite._graph([0, 1, 2,
3]).connected_components()),
      "component(s) and only", kite.L - 1, "loop: not the same integral")
```

```
U(kite) at x5 = 0 = U(kite with line 5 shrunk): True
F(kite) at x5 = 0 = F(kite with line 5 shrunk): True
the same graph with line 5 deleted instead has 1 component(s) and only 1 loop: not the same
integral
```

IN ONE LINE

A missing line costs nothing to cross. Its two ends act as one point and the diagram of the sector is the original one with that line shrunk.

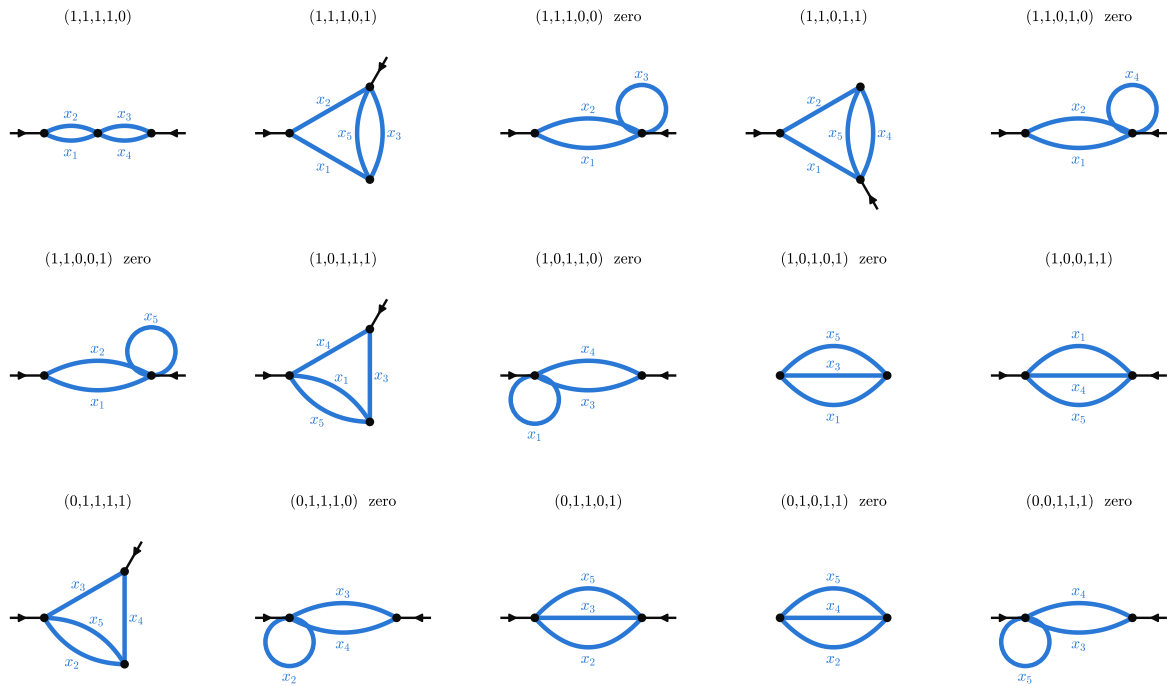
5.4 All sectors with four and three lines

`draw_sectors` draws each sector this way. The title is the sector and **zero** marks the sectors whose integrals all vanish (explained next).

```
In [36]: from itertools import product as cartesian
secs = [s for s in cartesian([1, 0], repeat=5) if sum(s) in (4, 3)]
titles = [str(s).replace(" ", "") + (" zero" if kfam.is_zero_sector(s) else "") for s in
secs]
print(sum(kfam.is_zero_sector(s) for s in secs), "of", len(secs), "are zero")
fig = draw_sectors(kite, secs, titles=titles, momenta=False, ncols=5, size=2.0)
```

```
8 of 15 are zero
```

Out[36]:



How to read them:

- The five four-line sectors are all non-zero. Shrinking line 5 gives the two bubbles in a row, as above. Shrinking any other line gives a triangle with a bubble on one side.
- Of the ten three-line sectors only two survive. Both are **sunsets**: three lines between the vertex where p enters and the vertex where it leaves.
- In every other three-line sector a line closes on itself at one vertex (a little loop), or both legs end up at the same vertex.

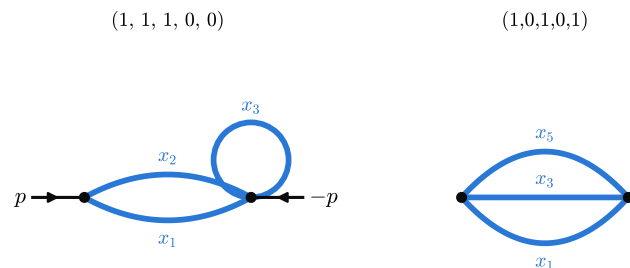
5.5 Zero sectors

Take the sector (1, 1, 1, 0, 0) and also (1, 0, 1, 0, 1):

```
In [37]: s0 = (1, 1, 1, 0, 0)
fig = draw_sectors(kite, [s0, (1, 0, 1, 0, 1)], titles=[str(s0), "(1,0,1,0,1)"],
momenta=True, ncols=2, size=2.4)
U0, F0 = kfam.UF(s0)
print("U =", U0, "    F =", F0)
print("zero sector (Lee's criterion):", kfam.is_zero_sector(s0))
```

```
U = x1*x3 + x2*x3      F = (-s)*x1*x2*x3
zero sector (Lee's criterion): True
```

Out[37]:



In (1, 1, 1, 0, 0) line 3 has become a loop that starts and ends at one vertex. Its loop momentum k appears in this line only, so the integral contains the factor

$$\int d^D k \frac{1}{k^2}$$

This has no scale at all: no mass and no external momentum. Under $k \rightarrow \lambda k$ it changes by λ^{D-2} , so it can only be zero in dimensional regularisation (the note shows this for the massless tadpole). One zero factor makes the whole sector zero. In $(1, 0, 1, 0, 1)$ both legs end up on the same vertex, where p comes in and goes out again. Their momenta cancel there, so the picture has no legs left: no momentum flows through the sunset, which is again a massless vacuum integral and zero.

BEYOND THE LECTURE: LEE'S CRITERION FOR ZERO SECTORS

The computer cannot look at pictures, so it uses a test of R. N. Lee (2013) on $G = \mathcal{U} + \mathcal{F}$ of the sector.

The test. The sector is zero if there are numbers k_i such that the rescaling $x_i \rightarrow \lambda^{k_i} x_i$ multiplies **every** monomial of G by the same factor λ .

Why it works. In the Lee-Pomeransky form of the parametric integral the integrand is $G^{-D/2}$ times powers $x_i^{a_i-1}$. Change variables $x_i \rightarrow \lambda^{k_i} x_i$. The integral cannot change (it is only a change of variables), but the integrand picks up a factor λ^c with $c = \sum_i k_i a_i - D/2$, which is not zero for general D . A number equal to λ^c times itself for every λ is zero.

In practice each monomial x^e of G gives one linear equation $\sum_i k_i e_i = 1$, and the sector is zero exactly when these equations have a solution. That is a few lines of linear algebra.

For $(1, 1, 1, 0, 0)$, $G = x_1 x_3 + x_2 x_3 - s x_1 x_2 x_3$ and $x_3 \rightarrow \lambda x_3$ does it ($k = (0, 0, 1)$): that is the lonely line 3. For the sunset no such rescaling exists:

```
In [38]: for s in [(1, 1, 1, 0, 0), (0, 1, 1, 0, 1), (1, 1, 1, 1, 0), (1, 1, 1, 1, 1)]:
          U0, F0 = kfam.UF(s)
          print(s, " U + F =", U0 + F0, " zero:", kfam.is_zero_sector(s))

(1, 1, 1, 0, 0) U + F = (-s)*x1*x2*x3 + x1*x3 + x2*x3 zero: True
(0, 1, 1, 0, 1) U + F = (-s)*x2*x3*x5 + x2*x3 + x2*x5 + x3*x5 zero: False
(1, 1, 1, 1, 0) U + F = (-s)*x1*x2*x3 + (-s)*x1*x2*x4 + (-s)*x1*x3*x4 + (-s)*x2*x3*x4 +
x1*x3 + x2*x3 + x1*x4 + x2*x4 zero: False
(1, 1, 1, 1, 1) U + F = (-s)*x1*x2*x3 + (-s)*x1*x2*x4 + (-s)*x1*x3*x4 + (-s)*x2*x3*x4 + (-
s)*x1*x2*x5 + (-s)*x2*x3*x5 + (-s)*x1*x4*x5 + (-s)*x3*x4*x5 + x1*x3 + x2*x3 + x1*x4 + x2*x4
+ x1*x5 + x2*x5 + x3*x5 + x4*x5 zero: False
```

Counted by the number of lines:

```
In [39]: from collections import Counter
allsec = [s for s in cartesian([0, 1], repeat=5) if sum(s) > 0]
nz = Counter(sum(s) for s in allsec if not kfam.is_zero_sector(s))
tot = Counter(sum(s) for s in allsec)
for k in sorted(tot):
    print("%d lines: %2d sectors, %2d non-zero" % (k, tot[k], nz.get(k, 0)))

1 lines:  5 sectors,  0 non-zero
2 lines: 10 sectors,  0 non-zero
3 lines: 10 sectors,  2 non-zero
4 lines:  5 sectors,  5 non-zero
5 lines:  1 sectors,  1 non-zero
```

Of the 31 sectors with at least one line only 8 are non-zero.

5.6 Why sectors matter

THREE FACTS THAT MAKE SECTORS USEFUL

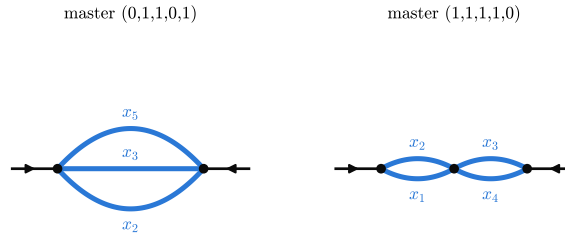
1. **IBP never creates a line.** An IBP identity can add a dot or a numerator, and a numerator can cancel a propagator, but no identity puts back a propagator that was absent. So an integral of one sector is always written through integrals of the **same or smaller** sectors. This is why Laporta's algorithm starts with the simplest sectors and works upwards.

2. **Zero sectors drop out at once.** Here 23 of the 31 sectors never need any work.
3. **Every master integral belongs to one sector.** For the kite the masters are one integral in the sunset sector and one in the bubble-bubble sector:

```
In [40]: red = ibp_reduce(kfam, ["F(1,1,1,1,1)"])
print("masters:", red.masters)
fig = draw_sectors(kite, [tuple(1 if x > 0 else 0 for x in m) for m in red.masters],
                    titles=["master " + str(m).replace(" ", "") for m in red.masters],
                    momenta=False, ncols=2, size=2.4)
```

```
masters: [(0, 1, 1, 0, 1), (1, 1, 1, 1, 0)]
```

Out [40]:



In [41]: red

Out [41]:

$$F(1, 1, 1, 1, 1) = \frac{(-2) \cdot (d-3)}{s \cdot (d-4)} F(1, 1, 1, 1, 0) + \frac{(2) \cdot (3d-10) \cdot (3d-8)}{s^2 \cdot (d-4)^2} F(0, 1, 1, 0, 1)$$

The kite itself, the top sector (1, 1, 1, 1, 1), has **no** master of its own: everything in it reduces to the two simpler sectors.

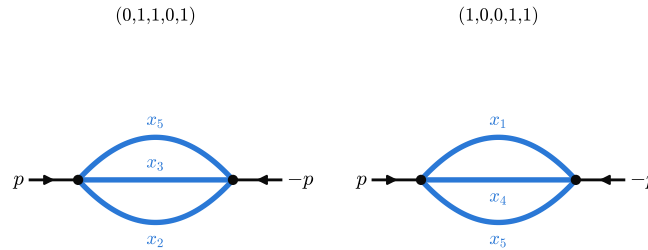
5.7 The same sector under another name

The kite has two non-zero sunset sectors, (0, 1, 1, 0, 1) and (1, 0, 0, 1, 1). Drawn, they are the same diagram: three massless lines between the in-vertex and the out-vertex. So their integrals are equal after renaming the lines and the reducer keeps only one of them. canon shows the representative it picks for a few integrals:

```
In [42]: from feynsage.laporta import Reducer
r = Reducer(kfam, symmetries=symmetries(kfam))
for a in [(1, 0, 0, 1, 1), (2, 0, 0, 1, 1), (1, 0, 0, 1, 3), (0, 1, 1, 0, 1)]:
    print("F%s -> F%s" % (a, r.canon(a)))
fig = draw_sectors(kite, [(0, 1, 1, 0, 1), (1, 0, 0, 1, 1)], titles=["(0,1,1,0,1)",
                                                                    "(1,0,0,1,1)"],
                    momenta=True, ncols=2, size=2.4)
```

```
F(1, 0, 0, 1, 1) -> F(0, 1, 1, 0, 1)
F(2, 0, 0, 1, 1) -> F(0, 1, 1, 0, 2)
F(1, 0, 0, 1, 3) -> F(0, 1, 1, 0, 3)
F(0, 1, 1, 0, 1) -> F(0, 1, 1, 0, 1)
```

Out[42]:



BEYOND THE LECTURE: SECTOR SYMMETRIES

Pak's criterion. An integral **without numerators** depends only on its powers a_i and on $G = \mathcal{U} + \mathcal{F}$ of its sector (that is all the parametric formula contains). So if renaming the x_i turns the G of one sector into the G of another, the two sectors hold the same integrals, line for line. Here are the two sunsets:

```
In [43]: for sec in [(0, 1, 1, 0, 1), (1, 0, 0, 1, 1)]:
          U0, F0 = kfam.UF(sec)
          print(sec, " U + F =", U0 + F0)

(0, 1, 1, 0, 1)  U + F = (-s)*x2*x3*x5 + x2*x3 + x2*x5 + x3*x5
(1, 0, 0, 1, 1)  U + F = (-s)*x1*x4*x5 + x1*x4 + x1*x5 + x4*x5
```

HOW THE COMPUTER COMPARES TWO POLYNOMIALS

Renaming $x_2, x_3, x_5 \rightarrow x_1, x_4, x_5$ turns one G into the other. To find such renamings for every pair of sectors at once, feynsage turns each G into a small graph (a vertex for each x_i , a vertex for each monomial coloured by its coefficient, an edge whenever x_i appears in a monomial) and asks Sage for its **canonical labelling**: a fixed way of numbering the vertices such that two graphs get identical labels exactly when they are the same up to renaming. Sectors with the same canonical form are the same; one of them is chosen as the representative and every integral is moved there. To keep this fast for large families, a cheap fingerprint (number of monomials, their coefficients and degrees) is compared first and only matching sectors are labelled.

For the kite the same renaming also follows from a symmetry of the whole diagram (the left-right reflection). Sometimes two sectors match **only** as sectors, with no symmetry of the full family behind them. The two-loop sunset with three equal masses (with the two numerator lines $k + p$ and ℓ that IBP needs) shows the difference:

```
In [44]: from feynsage.ff import reduce_ff
sun = family([("k", "1"), ("k - l", "1"), ("l + p", "1"), ("k + p", "1"), ("l", "1"), kin={"p^2": "pp"}])
targets = [(2, 1, 1, 0, 0), (2, 2, 1, 0, 0), (1, 1, 2, 0, 0)]
for ss in [False, True]:
    r = Reducer(sun, symmetries=symmetries(sun), sector_symmetries=ss)
    tab = reduce_ff(r, targets, rmax=2, smax=1)
    ms = sorted({m for row in tab.values() for m in row})
    print("sector symmetries %-5s: %d masters %s" % (ss, len(ms), ms))

sector symmetries False: 6 masters [(0, 1, 1, 0, 0), (0, 1, 1, 0, 1), (1, 0, 1, 0, 0), (1, 0, 1, 0, 1), (1, 1, 1, 0, 0), (1, 1, 2, 0, 0)]
sector symmetries True : 3 masters [(0, 1, 1, 0, 0), (1, 1, 1, 0, 0), (1, 1, 2, 0, 0)]
```

With sector symmetries the three tadpole-product sectors become one and the sunset sector keeps two masters: 3 in all, the same as Kira 3.1 finds (tests/test_kira_symmetries.sage compares every coefficient).

BEYOND THE LECTURE: INTEGRALS WITH NUMERATORS

With a numerator Pak's criterion is not enough: renaming the lines does not tell what happens to D_j upstairs. Then feynsage looks for a **shift of the loop momenta** $\ell \rightarrow A\ell + Bp$ (with $\det A = \pm 1$, so the

measure does not change, and the external momenta allowed to swap or change sign when their scalar products stay the same) that carries every propagator of the sector into the corresponding propagator of the other. Applied to the numerator, the shift turns D_j into a combination of propagators, which is written out. The result is one more linear equation between integrals, added to the IBP system. For example, for the equal-mass sunset, a symmetry inside the sunset sector gives

```
In [45]: r = Reducer(sun, symmetries=symmetries(sun), numerator_relations=True)
for rel in r.symmetry_relations((1, 1, 1, -1, 0)):
    print(" + ".join("(%s) F%s" % (c, a) for a, c in rel.items())).replace("+ (-", "- ("),
    "= 0")

(-1) F(0, 1, 1, 0, 0) - (1) F(1, 0, 1, 0, 0) - (1) F(1, 1, 0, 0, 0) + (2) F(1, 1, 1, -1, 0)
+ (1) F(1, 1, 1, 0, -1) - (pp - 3) F(1, 1, 1, 0, 0) = 0
```

These equations are added only when a master with numerators could need them; otherwise the reduction is left as it is.

6 Integration by parts and master integrals

This chapter follows the IBP part of the lecture and the course notebooks.

6.1 The bubble family

The equal-mass bubble of the note, Euclidean, $m = 1$. A massive line is written ("momentum", "mass").

```
In [46]: bubf = family(["l", "1"], ("l - p", "1"), kin={"p^2": "pp"}, euclidean=True)
bubf.info()

Integral family 'F'
  loop momenta: l   external momenta: p   Euclidean metric
  D1 = (l)^2 + 1
  D2 = (l - 1*p)^2 + 1
  An integral F(a1,...,at) is Int prod_r d^d l_r prod_i D_i^(-a_i); a_i <= 0 are
  numerators.
```

`ibp(a)` gives the identities for the seed $F(a)$, one for each vector (ℓ and p) inside the derivative. Each line reads $\sum c F = 0$ with d the dimension:

```
In [47]: for k, ident in enumerate(bubf.ibp((1, 1))):
    print("identity %d:" % (k + 1), " + ".join("(%s) F%s" % (c, a) for a, c in
    ident.items()), "= 0")

identity 1: (d - 3) F(1, 1) + (2) F(2, 1) + (pp + 2) F(1, 2) + (-1) F(0, 2) = 0
identity 2: (-pp) F(2, 1) + (1) F(2, 0) + (pp) F(1, 2) + (-1) F(0, 2) = 0
```

With $F(1, 2) = F(2, 1)$ the first one is the identity of the note, $0 = (D - 3)J(1, 1) + (p^2 + 4m^2)J(2, 1) - T(2)$.

6.2 Reduction to master integrals

`ibp_reduce(family, targets)` picks the seeds, finds the symmetries and solves the system:

```
In [48]: rb = ibp_reduce(bubf, ["F(2,1)", "F(2,2)"])
rb
```

Out[48]:

$$\begin{aligned}
 F(2,1) &= \frac{(-1) \cdot (d-3)}{(pp+4)} F(1,1) \\
 &\quad + \frac{(-\frac{1}{2}) \cdot (d-2)}{(pp+4)} F(0,1) \\
 [6pt] F(2,2) &= \frac{(-1) \cdot (d-2) \cdot (2d+pp-6)}{pp \cdot (pp+4)^2} F(0,1) \\
 &\quad + \frac{(d-3) \cdot (dpp-6pp-4)}{pp \cdot (pp+4)^2} F(1,1)
 \end{aligned}$$

The note has $J(2,1) = -(D-3)/(p^2 + 4m^2) J(1,1) + T(2)/(p^2 + 4m^2)$ and $T(2) = -(D-2)/(2m^2) T(1)$. With $m = 1$ both coefficients agree exactly:

```
In [49]: row = rb[(2, 1)]
dd, ppp = row[(1, 1)].parent().gens()
print(row[(1, 1)] == -(dd - 3)/(ppp + 4), row[(0, 1)] == -(dd - 2)/(2*(ppp + 4)))

True True
```

6.3 A two-loop number from two masters

The kite in the labelling of the note, $q^2 = 1$. Both masters are products of Gamma functions: the bubble times bubble is $G(1,1)^2$ and the sunset is $G(1,1)G(1,2-D/2)$, with $G(a,b)$ the massless one-loop bubble. Put them into the reduction and expand:

```
In [50]: from feynsage.oneloop import G, D, expand_eps
kq = family(["l1", "l1 + q", "l1 + l2", "l1 + l2 + q", "l2"], kin={"q^2": 1},
euclidean=True)
rk = ibp_reduce(kq, ["F(1,1,1,1,1)"])
value = {(0, 1, 1, 0, 1): G(1, 1)*G(1, 2 - D/2), (1, 1, 1, 1, 0): G(1, 1)^2}
kite_value = sum(SR(str(c)).subs({SR.var('d'): D}) * value[mm] for mm, c in rk[(1, 1, 1, 1, 1)].items())
expand_eps(kite_value, 0, loops=2)
```

Out[50]:

$$6\zeta(3)$$

All the poles and all the rational numbers cancel and $6\zeta(3)$ is left, the value of the note found there with Feynman parameters and Cheng-Wu.

6.4 Three reducers, one answer

BEYOND THE LECTURE: FINITE FIELDS

An IBP system for a real problem has hundreds of thousands of equations whose coefficients are rational functions of d and the invariants. Gaussian elimination with such coefficients swells: intermediate expressions become huge although the final answer is small. Modern programs (FIRE, Kira, FiniteFlow) avoid this:

1. Put numbers for d and the invariants, and do the arithmetic **modulo a large prime p** . Every number then fits in one machine word and nothing swells.
2. The first such run shows which equations the targets really need. All the others are thrown away (**trimming**).
3. Repeat at many values of d and rebuild each coefficient as a rational function of d from these values (Thiele interpolation, a continued fraction). With two primes, rebuild the rational numbers inside it (**rational reconstruction**) and check with a third.

feynsage does this in `method="ff"`, with a small C kernel for the first run. `method="trimmed"` uses the first run only to find the needed equations and then solves those with exact rational functions. `method="exact"` solves everything exactly, which is simple but slow. All three must agree:

```
In [51]: res = {meth: ibp_reduce(kq, ["F(2,2,1,2,2)"], method=meth) for meth in ["ff", "trimmed",
"exact"]}
for meth, r in res.items():
    print("%-8s %.2f s" % (meth, r.seconds))
```

```
print("the same coefficients:", res["ff"].table == res["trimmed"].table ==
res["exact"].table)

ff      0.11 s
trimmed 0.10 s
exact   4.44 s
the same coefficients: True
```

With `verbose=True` the finite-field reducer says how big the system was:

```
In [52]: r = ibp_reduce(kq, ["F(2,2,1,2,2)"], method="ff", verbose=True)

system: 6032 equations, 661 integrals (0.1s)
primes used: 2, total 0.1s
```

6.5 Seeds

`ibp_reduce` also chose the **seeds**: the integrals $F(a)$ for which the IBP identities were written. It takes every non-zero sector, adds up to `rmax` extra dots and `smax` numerator powers (enough for the targets) and orders the integrals Laporta's way: fewer lines first, then fewer dots and numerators. Each identity is solved for its most complicated integral. Whatever cannot be eliminated is a master integral. The note's Laporta chapter shows this by hand.

6.6 Counting masters without reducing

BEYOND THE LECTURE: THE LEE-POMERANSKY COUNT

Lee and Pomeransky (2013) found that the number of master integrals can be read off $G = \mathcal{U} + \mathcal{F}$ without any reduction: in each sector, set the missing x_j to zero and count the points where all derivatives of G vanish with every remaining $x_i \neq 0$. That number is the number of masters of the sector.

By hand for the bubble ($m = 1$): $G = x_1 + x_2 + x_1 x_2 p^2 + (x_1 + x_2)^2$. The two derivatives differ by $(x_2 - x_1)p^2$, so $x_1 = x_2 = x$ and $1 + x(p^2 + 4) = 0$: one point. In the tadpole sector $G = x_1 + x_1^2$ has one point too.

Two cautions (the master chapter of the note has the details): the count does not know about symmetries, so equal sectors must be counted once, and it needs the points to be isolated, which fails in some massless sectors.

A Gröbner basis counts the points. The extra variable t with $t \prod x_i = 1$ throws away the points where some $x_i = 0$:

```
In [53]: def critical_points(fam, sector, numbers):
    U, F = fam.UF()
    n = len(sector)
    R = PolynomialRing(QQ, ['x%d' % (i + 1) for i in range(n)] + ['t'])
    x, t = R.gens()[:n], R.gens()[n]
    G = R(str(SR(str(U + F)).subs(numbers)))
    live = [x[i] for i in range(n) if sector[i]]
    I = R.ideal([G.derivative(v) for v in live] + [x[i] for i in range(n) if not sector[i]]
    + [t*prod(live) - 1])
    return 0 if I.is_one() else I.vector_space_dimension()

pp = var('pp')
for sec in [(1, 1), (1, 0), (0, 1)]:
    print(sec, critical_points(bubf, sec, {pp: 7/3}))

(1, 1) 1
(1, 0) 1
(0, 1) 1
```

Top sector 1, each tadpole sector 1. The two tadpole sectors are the same integral, so $1 + 1 = 2$ masters, as IBP found.

7 Dirac traces with FORM

BEYOND THE LECTURE: FORM

FORM (J. Vermaseren) is the program in which most large perturbative calculations are done: it handles expressions with millions of terms. feynsage writes a short FORM program for each trace, runs it and reads the result back into Sage. Two conventions to know:

- 'g5' is γ_5 . With γ_5 the trace is done in 4 dimensions (FORM's `trace4`). In D dimensions γ_5 has no unique definition and one must choose a scheme, so feynsage does not mix γ_5 with D .
- FORM's Levi-Civita symbol `e_` is $-i$ times the usual $\epsilon^{\mu\nu\rho\sigma}$. feynsage prints it as `eps` and keeps FORM's convention.

The metric comes back as `g(mu, nu)`:

```
In [54]: form.trace(['mu', 'nu', 'rho', 'sigma'])
```

```
Out[54]: 4 g(mu, sigma) g(nu, rho) - 4 g(mu, rho) g(nu, sigma) + 4 g(mu, nu) g(rho, sigma)
```

A repeated index is summed ($\gamma^\mu \gamma_\mu = D$ in D dimensions). A factor like `'p + m'` means $\not{p} + m$:

```
In [55]: print(form.dirac_trace(['mu', 'mu'], vectors=[], dim='D'))
print(form.dirac_trace(['g5', 'mu', 'nu', 'rho', 'sigma'], vectors=[]))
print(form.dirac_trace(['p + m', 'mu', 'k + m', 'nu'], vectors=['p', 'k']))
```

```
4*D
4*eps(mu, nu, rho, sigma)
4*m^2*g(mu, nu) + 4*comp(k, nu)*comp(p, mu) + 4*comp(k, mu)*comp(p, nu) - 4*dot(p, k)*g(mu, nu)
```

7.1 A whole cross section: $e^+e^- \rightarrow \mu^+\mu^-$

For massless fermions summed over spins $\sum |\mathcal{M}|^2 = (e^4/s^2) \text{Tr}[\not{p}'\gamma^\mu \not{p}\gamma^\nu] \text{Tr}[\not{k}\gamma_\mu \not{k}'\gamma_\nu]$. Two traces share μ, ν , so we write the FORM program ourselves and run it with `form.run_form.g_(1, ...)` and `g_(2, ...)` are the two fermion lines. The `id` lines put in the Mandelstam variables, $p \cdot k = p' \cdot k' = -t/2$ and $p \cdot k' = p' \cdot k = -u/2$:

```
In [56]: code = '''
Symbols s, t, u;
Vectors p, pp, k, kp;
Indices mu, nu;
Local M = g_(1, pp, mu, p, nu) * g_(2, k, mu, kp, nu);
trace4, 1;
trace4, 2;
id p.k = -t/2; id pp.kp = -t/2;
id p.kp = -u/2; id pp.k = -u/2;
Print;
.end
'''

traces = SR(" ".join(form.run_form(code).split("M =")[1].split(";")[0].split()))
traces
```

```
Out[56]: 8 t^2 + 8 u^2
```

Average over the four spin states, go to the centre-of-mass frame ($t = -s(1 - \cos \theta)/2$, $u = -s(1 + \cos \theta)/2$) and use $d\sigma/d\Omega = |\mathcal{M}|^2/(64\pi^2 s)$:

```
In [57]: s, t, u, e, th, alpha = var('s t u e theta alpha')
msq = (e^4/s^2 * traces/4).subs(t=-s/2*(1 - cos(th)), u=-s/2*(1 + cos(th)))
dsig = (msq/(64*pi^2*s)).subs(e=sqrt(4*pi*alpha))
print("dsigma/dOmega =", dsig.trig_simplify().factor())
print("sigma =", integrate(integrate(dsig*sin(th), th, 0, pi), var('phi'), 0, 2*pi))
```

```
dsigma/dOmega = -1/4*(sin(theta)^2 - 2)*alpha^2/s
sigma = 4/3*pi*alpha^2/s
```

Since $2 - \sin^2 \theta = 1 + \cos^2 \theta$ this is $d\sigma/d\Omega = \alpha^2(1 + \cos^2 \theta)/(4s)$ and the total cross section is $4\pi\alpha^2/(3s)$, as in Peskin and Schroeder (eq. 5.12).

8 Where to go next

WHAT FEYNSAGE IS FOR

feynsage is meant for small and medium problems and for studying: every step is exact, can be checked and explains itself. On such problems it runs in the same range as the established tools and sometimes a little faster. For very large reductions (millions of equations, several scales) Kira and FIRE remain the better choice. I may continue improving feynsage in the future.

- github.com/aburousan/feynsage: the code, the issues page and the latest version.
- `tutorials/`: the same steps as five Jupyter notebooks, with exercises and solutions.
- `examples/feynsage_walkthrough.ipynb`: the longer tour, with the neutral pion lifetime and the QED of Chluba's thesis.
- `docs/DETAILS.md`: every module, every check against other programs and the speed numbers.
- The lecture note *Feynman integrals at one loop*: the physics behind every step.